Ternary CIM

首先大体的思路依然采用：5pack3，LUT，CIM，然后按照郭老师实现的大致的代码部分来，只不过cim 宏部分应该像下面这样直接调用组里面画好的一些基本CIM宏单元（是绑定好标准单元库的）（我后面会改一下这里），直接写verilog写出来的应该还是不能为CIM。目前我以下想的点还是以加速mpGEMM（W1.58A8）这一部分为主的。

```
module CimRow #(
    parameter int DEPTH = 16
)(
    input  var logic                      DIN, DINB, CKW, CKR,
    input  var logic [DEPTH - 1 : 0]      RWL, WWL,
    output var logic                      HOT
);

    wire WBL, WBLN, RBL;

    WD7T uWD7T(.DIN(DIN), .DINB(DINB), .CKW(CKW), .WBL(WBL), .WBLN(WBLN));

    RA6T uRA6T(.RBL(RBL), .CKR(CKR), .ACT(1'b1), .ACTN(1'b0), .HOT(HOT));

    for (genvar i = 0; i < DEPTH; i += 1) begin: SRAM
        S8T1 uS8T1(.RWL(RWL[i]), .WWL(WWL[i]), .WBLN(WBLN), .WBL(WBL), .RBL(RBL));
    end

endmodule

/*
    CimRow defines the basic unit of the CimArray.
    It contains a WD7T, a RA6T, and DEPTH S8T1 cells.
    WD7T is used to write the data to the CimArray.
    RA6T is used to read the data from the CimArray.
    S8T1 is used to store the data in the CimArray.

    The expected array size is 16*(8*32), DEPTH is 16. WIDTH is 8*32=256.
    The data is stored in the CimArray in the following order:
    [0:7] [8:15] [16:23] [24:31] [32:39] [40:47] [48:55] [56:63]
    [64:71] [72:79] [80:87] [88:95] [96:103] [104:111] [112:119] [120:127]
    [128:135] [136:143] [144:151] [152:159] [160:167] [168:175] [176:183] [184:191]
    [192:199] [200:207] [208:215] [216:223] [224:231] [232:239] [240:247] [248:255]
*/
```

Main innovations list below:

1. *5-pack-3 Zero-Aware Encoding*

Unpack			Pack
-1	-1	-1	1 1101
-1	-1	0	1 1100
-1	-1	1	1 1011
-1	0	-1	1 1010
...			
0	0	0	0 0000
...			
1	0	1	0 1010
1	1	-1	0 1011
1	1	0	0 1100
1	1	1	0 1101

上图是TL2内核对应的package形式，是纯查表逻辑，直接根据pack后的后4bit（weight index）进行查找LUT然后根据1bit（weight sign）进行符号翻转（mirror）。

但这样package的方式虽然足够高效（32存27），且一定程度上对内存alignment更友好，但是却容易忽略包装起来的一个group内的weight的稀疏性，以及并不小的LUT的构建成本。（可以预想如果一个Word line上装载有128个8T SRAM cells，那么将对应一次构建32个14entry的Look Up Table，成本也比较大）

因此我想更换一种package的encoding方式，能够更高效的表示出当前group里0的个数，非零值（零值）的位置，以便后续能更好的利用。具体的encoding方式如下，仍然遵从第一位为符号位的设定进行mirror，也就是主要改动后4bit如何encode14种状态：

Weight			Pack	Description
			00--	0个0 *4
			01--	1个0（无负的）*3
			10--	1个0（有负的）*3
			11--	2个0 *3
0	0	0	1111	3个0 *1
Pack的第一位依旧是符号位，进行mirror。				

没写全，先简单写了。这样的encoding方式看起来是更能够说明其中的element-wise的sparsity的，也能更specific说明zero-distribution，直观上感觉也会更能够容易优化。要比原本的按顺序从小

到大排布（maybe可能也许）更好一些（？）。

2. *Hierarchy Dynamic SM LUT*

其实整 个Bitnet1.58网络权重中0的比例是很高的，这也正是1.58b相对bitnet的一个很大的优势。所以我想在构建LUT的时候，没有必要全部都构建，我们只构建9个entry（除去一个全零entry，除去4个全非零entry）。这里我们做了一个假设那就是在bitnet1.58的权重排列中，全非零这种group的出现的概率并不那么高，因此我们只构建这9种LUT，也就是根据encoding，如果是1111，那么直接跳过该LUT，不产生任何输出（输出0）；如果encoding是00开头，那么直接动态的根据其他已有的entry现算当前对应的entry（这四个情况对应的值都可以根据其他算好的entry两两加和得到，所以本质上是进行一次组合plus运算），然后输出对应的结果到后续的adder tree。这样可以节省36%的LUT比例，基本不需要额外的电路，多的一次计算相比于构建LUT的计算也少很多，应该会有优化。

3. *Sign-Magnitude Dual Adder Tree*

Reference:

https://ieeexplore.ieee.org/document/10067269/

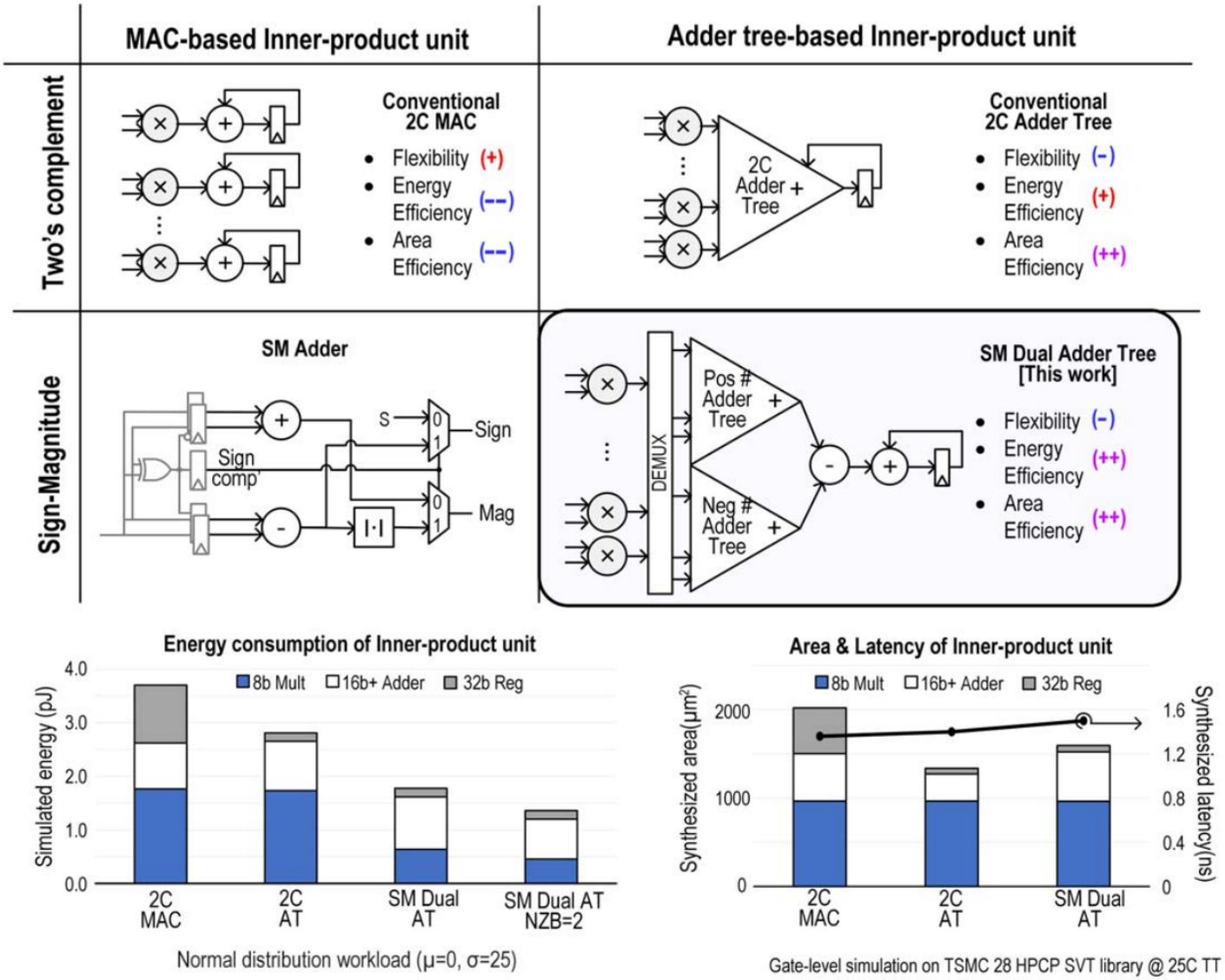
29.3 An 8.09TOPS/W Neural Engine Leveraging Bit-Sparsified Sign-Magnitude Multiplications and Dual A

The computational complexity of neural networks (NNs) continues to increase,…

这里主要考虑了23年的一篇工作，我觉得可以是一个加法器优化的点，这篇工作主要claim了一种更低翻转低功耗的adder tree 策略，具体就是用sign-magnitude的方式粗暴的取代原有的2C编码格式，然后将有符号的adder tree变为两个无符号的tree，最后再进行一步subtract。应该会有能耗的reduction。

胡老师今年的一篇前不久的MICRO也采用了这个方法（3D-PATH那篇），claim了类似的事情。

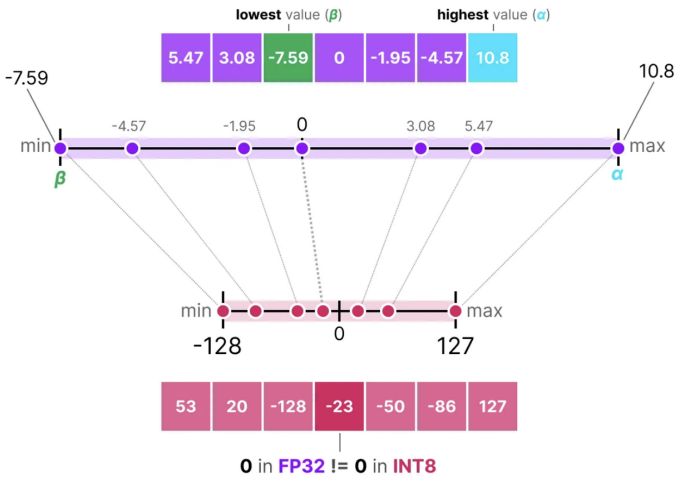
只不过我们想把它跟我们的LUT套起来可能还需要一些别的改动。因为我们输入的activation是有符号的8bit值（2C形式），所以有两种方案可以套进去，第一种是在构建LUT的时候就动手，使得最终构建的LUT的每一个entry都是{current_entry_sign, current_entry_magnitude}的形式，这样再结合group_sign，就能判定查表读出来的每一个值是去POS tree还是NEG tree。第二种就是正常2C构建表格，得到每个entry是正常的表示，然后再做一些逻辑组合电路把值分配到POS tree或者NEG tree。



4. *Asymmetric-supported design*

对于ternary的LLM来讲，本身呢就会存在一些accuracy的loss，因此推理的精度需要尽可能的高一些。对于INT8的activation，如果采用symmetric quantization的话，虽然相对易于计算，但由于不同的activation可能带来比较大的distribution的偏移，比如mostly positive的话，那么INT8表达能力就没有被完全的发挥出来。

所以我想支持对于Asymmetric quantization的支持，（比如右图这种，实际操作时候会离线做PTQ，calibaration得到zero-point），这种量化带来了更精准的分布，也能充分的发挥INT8的表达能力，问题就是带来了zero-point这样的偏移，会使得计算变得复杂。



比方说我们的zero point是85，那么用二进制的话就是01010101，靠近该zero-point附近的值应该有很多（因为整体的数据分布都是呈现类似正态的分布，中间多，两头少），所以可以大致认为很多值都是0101xxxx，因此如果我们能对

这样格式的数据计算优化的话，应该会有增益。
能对非对称量化有更好的支持。

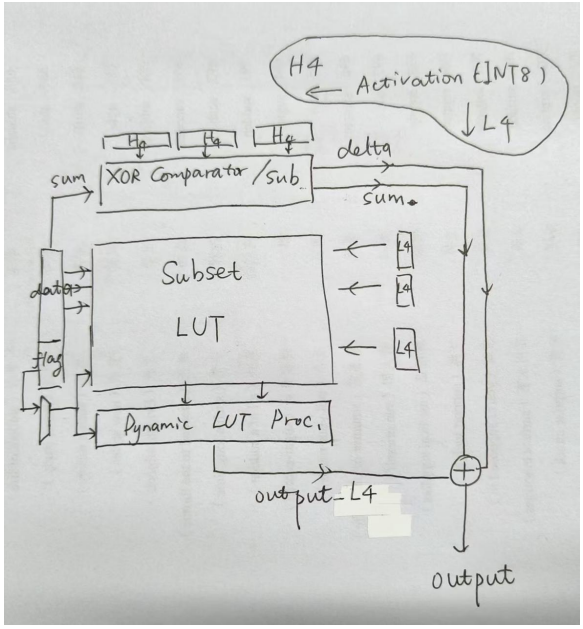
我们考虑INT8分为两部分，高4bit和低4bit。

那么顺利成章我们可以把一个数和8bit数的乘法转换为低4bit乘法和高4bit乘法（需要shift4位）两部分。而如果很多数的高四位都是zero point这样的0101，那么是不是这部分可以不进入LUT，只需要通过补偿就可以了从而简化建表的大小，进而显著减少动态功耗和面积呢？

Encoding Strategy

sign	flag	data	weight	sum	
0	0	000	000	0	Needed
0	0	001	001	1	
0	0	010	010	1	
0	0	011	011	2	
0	0	100	100	1	
0	0	101	101	2	
0	0	110	110	2	
0	0	111	111	3	
0	1	000	01-1	0	Free(Dynamic)
0	1	001	10-1	0	
0	1	010	1-10	0	
0	1	011	1-11	1	
0	1	100	11-1	1	
0	1	101	-111	1	
indicate the sign.	whether in LUT+whether contains negatives.	encoding	weight	popcount of weight, used for high 4 bits compensation.	Description
5-bit encoding			***actual***	***addition logic***	

Encoding Strategy



Ugly Ugly architecture

那我们不妨考虑一下对于我们这种ternary的pack情况，对于每一个LUT的构建会输入进来三个activation。最好的情况就是三个activation的high4bit（记作H4了）都是ZP。

根据这个encoding的编码方式，最终执行计算可能如下：

首先，输入了3个activation，举例两种情况

1. 01010110，01010101，01011111

应该很多都会是这种情况，那么这种情况，判断三个数的H4===ZP，存LUT的时候只对L4存LUT，且只对橙色区的8个存，这样的话entries数只有8，而且每个entry小很多bit，应该6bit就够。绿色的部分依然是之前讨论的dynamic的形式。

然后weight来的时候，比方说weight是0111（不考虑sign），那么就是正常选LUT，然后额外要加上

$$(Sum \ * \ ZP) >> 4$$

这样的话就得到了结果。readout。

2. 01010011，00001111，01011111

仅有两个是ZP开头的，那么这时候做的事情就是，先把不等于0101的做减法得到delta，然后最后加上

$$(Sum \ \times \ ZP) >> 4 \ + delta >> 4$$

通过这样的方式，我们能充分利用了zero point，且能保证asymmetric的精度得以发挥，最大化的减少不对称量化带来的计算复杂度的增加，同时有效的减少了LUT的构建。

不知道是不是很严谨也不确定能不能带来性能的提升捏～～～～～～～～～～～～